

Performance and Robustness Benchmarking Across Battery State-of-Charge Estimator Classes for Embedded Applications

Florian Rzepka*, Qinnan Chen, Julia Kowal

Technical University of Berlin, Chair of Electrical Energy Storage Technology,
Einsteinufer 11, 10587 Berlin, Germany

*Corresponding author: Florian Rzepka, florian.rzepka@tu-berlin.de

Qinnan Chen: qinnan.chen@campus.tu-berlin.de

Julia Kowal: julia.kowal@tu-berlin.de

Abstract

Embedded battery-state estimators require validation under disturbed inputs and the inference semantics used on resource-constrained hardware. This study presents a causal, cell-disjoint benchmark of four State-of-Charge (SOC) approaches: fixed-capacity Coulomb counting as a direct-measurement model (DM), Coulomb counting with data-driven State-of-Health adaptation as a hybrid direct-measurement model (HDM), a second-order resistor-capacitor observer as a hybrid equivalent-circuit model (HECM), and a gated recurrent unit structurally pruned from 96 to 67 hidden units as the data-driven model (DD). Public lithium iron phosphate ageing data cover multiple operating and health conditions. Robustness evaluation addresses nominal accuracy, corrupted or unavailable measurements, and recovery from initialization errors. Hardware evaluation on an STM32H753 measures numerical equivalence, latency, flash, and peak runtime memory. For DD, exact rolling-window recomputation, continuous state forwarding, and periodic state reset are compared. Nominal cell-macro mean absolute errors are 0.0690, 0.0412, 0.0337, and 0.0258 for DM, HDM, HECM, and DD. DD yields the lowest nominal error and leads several disturbance scenarios, whereas HECM shows the strongest recovery. All implementations reproduce their software references, fit available memory, and meet the one-second sampling deadline. Compared with rolling-window recomputation, continuous-state DD reduces median latency from approximately 724 ms to 0.425 ms and peak memory from 67.3 KiB to 4.1 KiB while retaining similar nominal error. Periodic resets retain these resource benefits but cause pronounced transients. The results demonstrate that estimator choice, model compression, and recurrent-state handling must be evaluated jointly with accuracy, robustness, recovery, and embedded cost.

Keywords: State-of-charge estimation; Battery management system; Recurrent neural network; Robustness benchmarking; Embedded artificial intelligence; Model compression

Funding

This research was funded by the German Federal Ministry of Education and Research (BMBF), grant number 03SF0684A (MG FARM). The authors are responsible for the content. The work was further supported by the German Research Foundation and the Open Access Publication Fund of the Technical University of Berlin.

Acknowledgements

The authors acknowledge the High Performance Computing Service of the Technical University of Berlin for providing the computational resources used for training, benchmarking, and figure generation.

CRedit authorship contribution statement

Florian Rzepka: Conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing—original draft preparation, writing—review and editing, visualization. **Qinnan Chen:** Writing—review and editing. **Julia Kowal:** Resources, supervision, project administration, funding acquisition, writing—review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used ChatGPT, DeepL, and Grammarly to improve the linguistic quality and style of the manuscript. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Data availability

The dataset is publicly available through its cited TU Berlin repository record. Code, model configurations, benchmark definitions, random seeds, result summaries, hardware timing records, and analysis procedures are maintained at https://github.com/FRzepka/1_SCRIPTS.